

SOAL TEST BACKEND ENGINEER

Microservice Architecture - Gateway, Auth, User

Durasi pengerjaan: maksimal 3 hari kalender

Output: Repository Git, dokumentasi, dan instruksi run/deploy

Tujuan test ini adalah menilai kemampuan kandidat dalam merancang dan membangun backend berbasis microservice, menerapkan autentikasi, mengelola data menggunakan PostgreSQL dan MongoDB, membuat dokumentasi API, serta menyiapkan aplikasi agar siap dijalankan atau dideploy.

Catatan untuk kandidat: fokus utama penilaian adalah kualitas desain, kerapian kode, keamanan dasar, dokumentasi, dan kemudahan menjalankan aplikasi.

1. Deskripsi Tugas

Buat sebuah sistem backend dengan arsitektur microservice. Sistem minimal terdiri dari tiga service utama: Gateway Service, Auth Service, dan User Service. Kandidat boleh menggunakan bahasa pemrograman Golang atau NestJS.

Tema aplikasi bebas, namun seluruh fitur yang diminta harus tersedia. Contoh tema yang dapat digunakan: user management, employee management, member management, atau simple admin system.

Service yang wajib dibuat

- **Gateway Service** - menjadi pintu masuk semua request dari client dan meneruskan request ke service internal yang sesuai.
- **Auth Service** - menangani register, login, validasi token, refresh token opsional, dan otorisasi dasar.
- **User Service** - menangani data user/profile, termasuk CRUD user dan pencarian/filter sederhana.

2. Stack Teknologi

- Backend menggunakan salah satu pilihan: **Golang** atau **NestJS**.
- Database wajib menggunakan **PostgreSQL** dan **MongoDB**.
- Dokumentasi API wajib menggunakan **Swagger/OpenAPI**.
- Disarankan menggunakan Docker dan Docker Compose untuk menjalankan semua service dan database secara lokal.
- Deployment bersifat nilai tambah. Kandidat boleh deploy ke VPS, Railway, Render, Fly.io, Google Cloud Run, AWS, atau platform lain.

3. Requirement Fungsional

Fitur minimal yang harus tersedia:

Auth Service

- Register user baru dengan validasi input.
- Login menggunakan email/username dan password.
- Password harus di-hash, tidak boleh disimpan dalam bentuk plain text.
- Generate access token berbasis JWT.
- Endpoint validasi token untuk memastikan token masih valid.
- Role sederhana, minimal user dan admin.

User Service

- Create user/profile.
- Get list user dengan pagination.
- Get detail user berdasarkan ID.
- Update user/profile.
- Delete user atau soft delete user.

- Filter/search user berdasarkan nama, email, atau role.

Gateway Service

- Menerima semua request dari client melalui satu base URL.
- Meneruskan request ke Auth Service dan User Service.
- Menerapkan middleware autentikasi untuk endpoint yang membutuhkan login.
- Menangani error response standar dari service internal.

4. Requirement Database

Gunakan PostgreSQL dan MongoDB dalam sistem. Kandidat bebas menentukan service mana yang menggunakan database tertentu, selama keduanya digunakan secara jelas dan relevan.

- Contoh penggunaan PostgreSQL: data akun, credential, role, dan relasi utama user.
- Contoh penggunaan MongoDB: user profile, activity log, audit log, metadata, atau data fleksibel lainnya.
- Sertakan migration atau auto-schema setup untuk PostgreSQL bila memungkinkan.
- Sertakan seed data opsional untuk mempermudah reviewer melakukan testing.

5. Requirement API dan Dokumentasi

- Semua endpoint wajib terdokumentasi di Swagger/OpenAPI.
- Swagger minimal menampilkan method, path, request body, response success, dan response error.
- Sertakan cara mengakses Swagger pada README, misalnya /docs atau /api/docs.
- Gunakan format response yang konsisten untuk success dan error.
- Sertakan contoh request dan response untuk endpoint utama.

6. Requirement Non-Fungsional

- Kode harus rapi, modular, dan mudah dibaca.
- Gunakan environment variable untuk konfigurasi seperti database URL, JWT secret, port, dan service URL.
- Jangan commit secret asli ke repository.
- Gunakan validasi input pada endpoint yang menerima request body atau query parameter.
- Terapkan error handling yang konsisten.
- Terapkan logging dasar pada setiap service.
- Sertakan health check endpoint pada setiap service, misalnya /health.

7. Endpoint Minimal

Service	Method	Endpoint	Keterangan
Gateway	GET	/health	Health check gateway
Auth	POST	/auth/register	Register user baru
Auth	POST	/auth/login	Login dan mendapatkan JWT
Auth	GET/POST	/auth/validate	Validasi token

Service	Method	Endpoint	Keterangan
User	GET	/users	List user dengan pagination dan filter
User	GET	/users/{id}	Detail user
User	POST	/users	Create user/profile
User	PUT/PATCH	/users/{id}	Update user/profile
User	DELETE	/users/{id}	Delete atau soft delete user

8. Deliverables

- Repository Git berisi seluruh source code.
- README yang menjelaskan arsitektur, cara install, cara menjalankan lokal, dan cara menjalankan test.
- File .env.example untuk setiap service atau satu file env utama jika menggunakan Docker Compose.
- Dockerfile untuk setiap service, jika menggunakan Docker.
- docker-compose.yml untuk menjalankan gateway, auth, user, PostgreSQL, dan MongoDB secara lokal.
- Dokumentasi Swagger/OpenAPI yang dapat diakses saat service berjalan.
- Postman collection atau curl examples bersifat opsional tetapi menjadi nilai tambah.
- URL deployment jika aplikasi berhasil dideploy.

9. Ketentuan Deployment

Deployment tidak wajib, tetapi sangat disarankan dan akan menjadi nilai tambah. Jika dideploy, kandidat perlu memastikan minimal Gateway Service dapat diakses publik dan dokumentasi Swagger dapat dibuka oleh reviewer.

- Sertakan base URL deployment di README.
- Sertakan instruksi testing endpoint pada environment production/staging.
- Pastikan tidak ada credential sensitif yang terekspos di repository atau Swagger.
- Jika hanya sebagian service yang dideploy, jelaskan batasannya secara transparan di README.

10. Kriteria Penilaian

Aspek	Bobot	Indikator Penilaian
Arsitektur Microservice	20%	Pemisahan service jelas, komunikasi antar-service rapi, gateway berfungsi sebagai entry point.
Implementasi Fitur	20%	Auth, user management, validasi token, pagination/filter, dan role berjalan sesuai requirement.
Database Design	15%	PostgreSQL dan MongoDB digunakan dengan tepat, struktur data jelas, migration/seed menjadi nilai tambah.
Code Quality	15%	Kode modular, konsisten, mudah dibaca, error handling dan logging baik.
Security Basic	10%	Hash password, JWT secret via env, validasi input, endpoint protected, tidak expose secret.
Dokumentasi	10%	README lengkap, Swagger jelas, instruksi run mudah diikuti.

Aspek	Bobot	Indikator Penilaian
Docker/Deployment	10%	Docker Compose berjalan baik, deployment tersedia, konfigurasi production diperhatikan.

11. Bonus Point

- Unit test atau integration test untuk endpoint penting.
- CI/CD sederhana menggunakan GitHub Actions atau sejenisnya.
- Rate limiting pada Gateway Service.
- Refresh token flow.
- Role based access control yang lebih lengkap.
- Observability sederhana seperti request ID, structured logging, atau tracing.
- Caching menggunakan Redis.
- API versioning, misalnya /api/v1.

12. Catatan Pengumpulan

- Kirim link repository Git yang dapat diakses reviewer.
- Pastikan README berada di root repository.
- Pastikan aplikasi dapat dijalankan hanya dengan mengikuti instruksi pada README.
- Jika ada requirement yang belum selesai, tuliskan bagian yang belum selesai dan alasannya.
- Cantumkan asumsi teknis yang digunakan selama pengerjaan.

Format pengumpulan yang disarankan:

Nama Kandidat:

Posisi:

Link Repository:

Link Deployment (jika ada):

Catatan Tambahan: